

# **Smart Grid Optimization Using Machine Learning**

by

(TEAM-E)

(Lokesh Setty Kalakonda,  
Kesari Nandan Malleshally,  
Ajay Kumar Reddy Kethireddy)

**FINAL PROJECT REPORT**

for

**DATA 606 Capstone in Data Science**

**University of Maryland Baltimore County**

**2025**

Copyright ©2025  
ALL RIGHTS RESERVE

## ABSTRACT

Electricity load forecasting plays a critical role in power providers' energy planning, grid stability, and operational efficiency. This project aims to create interpretable and accurate load forecasting models by merging historical time-series approaches with sophisticated feature engineering based on weather conditions. With historical electricity generation data and real weather data from 2020 up to early 2025, we crafted a comprehensive dataset that included smart features like lagged load values, rolling averages, holiday indicators, as well as seasonal and hourly time indicators. The primary modeling framework leverages Facebook Prophet, an industry-leading forecasting library that handles complex seasonal patterns with ease, bolstered by custom regressors based on weather data. Other evaluated models include linear regression, XGBoost, and ensemble machine learning techniques to strengthen validation. The final model achieved a high predictive power with low error metrics while effortlessly estimating electricity load for a blind forecast window using only real weather data alongside engineered features. This research illustrates the outstanding integration of niche domain know-how, meteorological data, advanced modeling techniques, and their collaboration toward producing scalable infrastructure for sophisticated energy demand forecasting.

## ACKNOWLEDGMENTS

We would like to express our sincere gratitude to Dr. Unal Sakoglu, our instructor for the DATA 606 Capstone course, for his invaluable guidance and support throughout the course of this project. His insightful feedback, encouragement, and expertise in applied data science were instrumental in helping us refine our approach and effectively tackle the challenges of load forecasting.

We would also like to extend our appreciation to the organizations that provided open and accessible datasets crucial to this work. The **California Independent System Operator (CAISO)**, through their public portal on grid operations and load data ([caiso.com](https://www.caiso.com)), served as a vital source for historical electricity production data. Additionally, we are grateful to **Visual Crossing Weather** ([visualcrossing.com](https://www.visualcrossing.com)) for offering detailed and accurate hourly weather data, which allowed us to enrich our model with environmental factors.

Finally, we thank our peers, friends, and families for their continuous encouragement and motivation throughout the duration of this project. Their support helped us stay focused and inspired as we explored, built, and evaluated predictive models for real-world forecasting.

This project has been a challenging and rewarding experience, and we are genuinely appreciative of everyone who contributed to its success.

## TEAM MEMBERS CONTRIBUTIONS

This project was a collaborative effort undertaken by a team of three members, each contributing their unique skills and expertise to ensure the successful completion of the electricity load forecasting system.

| Name                               | Duties                                                                                                                                                                                                                                                                                            | Achievements                                                                                                                                                                                                                          |
|------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Lokesh Setty Kalakonda</b>      | Focused on raw data cleaning and alignment between Production and Weather datasets. Built machine learning pipelines(Linear Regression, Decision Trees, XGBoost), performed hyperparameter tuning, and benchmarked them against the Prophet model.                                                | Developed and fine-tuned ML models including Linear Regression, Decision Trees, and XGBoost. Successfully benchmarked Prophet's performance against ensemble methods with detailed evaluation metrics.                                |
| <b>Ajay Kumar Reddy Kethireddy</b> | Conducted exploratory data analysis (EDA), created visualization dashboards to compare actual vs predicted loads, and handled component decomposition plots. Contributed significantly to the final documentation, presentation visuals, and insights interpretation.                             | Produced clear and insightful visualizations of load patterns across time dimensions (hourly, weekly, yearly). Played a key role in interpreting model components and structuring the final report for clarity and impact.            |
| <b>Kesari Nandan Mallepally</b>    | Led the overall modeling strategy using Prophet, including feature engineering and regressor selection. Developed lag-based and weather-informed features, implemented time-series forecasts for future periods, evaluated model performance with key metrics, and interpreted seasonal patterns. | Successfully forecasted load for unseen data using real weather data with smart regressors with an impeccable R2-score of approximately 0.9831. Demonstrated Prophet's ability to model seasonality and trend with custom regressors. |

## TABLE OF CONTENTS

|                                                                  |      |
|------------------------------------------------------------------|------|
| ABSTRACT .....                                                   | iii  |
| ACKNOWLEDGMENTS .....                                            | iv   |
| TEAM MEMBERS CONTRIBUTIONS .....                                 | v    |
| LIST OF TABLES .....                                             | vii  |
| LIST OF FIGURES.....                                             | viii |
| I. INTRODUCTION .....                                            | 1    |
| I.1 Objective                                                    |      |
| I.2 Background & Literature Review/Survey                        |      |
| I.3 Datasets                                                     |      |
| II. METHODS .....                                                | 4    |
| II.1 Data Processing and Feature Engineering                     |      |
| II.2 Forecasting Model Analysis                                  |      |
| III. RESULTS .....                                               | 7    |
| IV. DISCUSSIONS AND CONCLUSIONS .....                            | 10   |
| V. FUTURE WORK .....                                             | 10   |
| VI. REFERENCES.....                                              | 11   |
| VII. APPENDIX .....                                              | 12   |
| VII.1 Programming Codes                                          |      |
| VII.2 Pictures/Snapshots of Data /Videos, Supplementary Material |      |

## LIST OF TABLES

|                                             |    |
|---------------------------------------------|----|
| Table 1. Model Evaluation Comparision ..... | 7  |
| Table 2. Prophet (actual vs predicted)..... | 14 |

## LIST OF FIGURES

|                                              |    |
|----------------------------------------------|----|
| Fig 1 Prophet Forecasting .....              | 7  |
| Fig 2 Linear Regression Forecasting .....    | 8  |
| Fig 3 Polynomial Regression Forecasting..... | .8 |
| Fig 4 Decision Tree Forecasting .....        | .8 |
| Fig 5 Bagging Regression Forecasting .....   | .8 |
| Fig 6 Stacking Regression Forecasting.....   | .8 |
| Fig 7 XGBoost Forecasting.....               | .8 |
| Fig 8 First Two Weeks Load Prediction .....  | 15 |
| Fig 9 Last Two Weeks Load Prediction .....   | 15 |
| Fig 10 Feature vs Coefficient Values.....    | 16 |
| Fig 11 Correlation Matrix .....              | 17 |
| Fig 12 Daily Load Pattern Forecasted.....    | 18 |
| Fig 13 Weekly Load Pattern Forecasted .....  | 18 |
| Fig 14 Yearly Load Pattern Forecasted.....   | 19 |

## I. Introduction

The increasing integration of renewable energy sources into power grids has made electricity load forecasting more critical than ever. Accurate short-term demand forecasting is essential for balancing supply and demand, optimizing resource allocation, and ensuring system stability in smart grid environments. In this project, we aim to build a robust forecasting model for hourly electricity load using a combination of time-series modeling and weather-informed features.

We utilized a dataset spanning from 2020 to January 2025, combining historical load data from the California Independent System Operator (CAISO) with high-resolution hourly weather data from Visual Crossing. The combined dataset enabled us to incorporate temporal and environmental factors influencing energy demand.

A core component of our approach is the use of Facebook Prophet, a time-series forecasting model that captures daily, weekly, and yearly seasonality and allows for the inclusion of custom external regressors. We enhanced this model with engineered features such as lagged load values, rolling means, working hour flags, and meteorological variables like temperature, solar energy, humidity, and sea-level pressure.

The dataset was split into training (80%) and testing (20%) sets, with the testing period primarily covering the year 2024 and January 2025. Our goal was to predict electricity load during January 2025 using models trained exclusively on historical data. This allowed for a realistic assessment of how well the model generalizes to recent unseen conditions while comparing predictions against known actuals.

To ensure robustness, we also benchmarked Prophet's performance against traditional machine learning models, including linear regression, decision trees, ensemble techniques, and XGBoost, evaluating each model using MAE, RMSE, MAPE, and  $R^2$  metrics.

This project demonstrates how effective feature engineering and hybrid modeling strategies can enhance the accuracy of electricity load forecasting and support operational planning in modern smart grids.

### I.1 OBJECTIVE

The primary goals of this project include:

- 1. Build an Accurate Time-Series Model for Load Forecasting:**

Use Facebook Prophet to model electricity load with hourly resolution by capturing key seasonal trends and including weather and temporal regressors.

- 2. Engineer Smart Features to Enhance Forecast Quality:**

Incorporate features such as lagged load values, rolling averages, hour-of-day, weekend flags, holidays, temperature variation, and solar energy to better represent demand-driving factors.

- 3. Evaluate Model Performance on Unseen Data:**

Assess the model's ability to generalize by testing it on a held-out portion of recent data, simulating a realistic deployment scenario in which the model must predict load based on previously unseen yet verifiable conditions.

## **I.2.1 BACKGROUND**

Electricity load forecasting is a foundational task in modern power systems, enabling utilities and grid operators to plan generation, reduce operational risks, schedule maintenance, and maintain supply-demand balance. As smart grids grow more complex with the integration of intermittent renewable sources like solar and wind, accurate and adaptable load prediction becomes critical for grid stability and efficiency.

Load patterns are influenced by numerous factors, including time-based variables (hour of day, day of week, season), consumer behavior, and external drivers such as weather conditions (temperature, humidity, solar radiation). Traditional forecasting methods often fall short in handling the complex, nonlinear, and multi-seasonal nature of modern load data.

To address these challenges, modern forecasting approaches increasingly incorporate data-driven techniques capable of capturing both long-term trends and short-term variations, while being flexible enough to include external regressors such as weather features.

## **I.2.2 LITERATURE REVIEW**

Historically, techniques such as ARIMA (Autoregressive Integrated Moving Average) and exponential smoothing were widely used for load forecasting due to their simplicity and ability to model time-series behavior. However, these classical methods often assume linearity, stationarity, and lack the flexibility to include multiple exogenous variables.

To overcome these limitations, a variety of machine learning models have been explored. These include Support Vector Regression (SVR), Random Forests, Gradient Boosted Trees (e.g., XGBoost), and ensemble learning methods. These models can handle non-linear interactions and take advantage of a richer set of engineered features, including weather-based variables and time-based flags. Ensemble approaches often improve generalization and have been found effective in various demand forecasting competitions and industrial studies.

In parallel, deep learning models, especially LSTM (Long Short-Term Memory) and GRU (Gated Recurrent Unit) networks, have shown strong potential in modeling sequential dependencies and long-term temporal patterns in energy data. Despite their power, these models require large volumes of data, are resource-intensive, and often suffer from a lack of interpretability — a drawback in critical infrastructure applications.

In recent years, Facebook Prophet has emerged as a promising alternative that balances interpretability with forecasting performance. Prophet is designed to handle multiple seasonality components (daily, weekly, yearly), holiday effects, changepoints, and missing values. It allows the inclusion of custom external regressors, making it highly suitable for load forecasting tasks where weather and calendar effects significantly influence demand.

Several studies in the energy domain have demonstrated that integrating weather features- such as temperature, humidity, solar radiation, wind speed, and pressure- significantly improves the forecasting accuracy of both statistical and machine learning models. This project builds upon that foundation by combining Prophet with smart feature engineering to create a hybrid framework capable of accurate and scalable short-term electricity load forecasting.

### I.3 Datasets

The dataset used in this project is the result of merging two distinct hourly datasets covering the period from **January 1, 2020 to January 31, 2025**, totaling **44,592 rows**. The first dataset was obtained from [CAISO \(California Independent System Operator\)](#) and includes comprehensive production-related variables. This production data consists of features such as total system **load**, **solar**, **wind**, **thermal**, **nuclear**, **imports**, **large hydro**, **net load**, **renewables**, and **generation**, along with an auxiliary feature named "*load less (generation + imports)*" to measure discrepancies between consumption and supply.

The second dataset, obtained from [Visual Crossing](#), provides **weather attributes** for California at the same hourly resolution. These include **temperature**, **dew point**, **humidity**, **precipitation**, **wind gust**, **wind speed**, **wind direction**, **sea level pressure**, **cloud cover**, **visibility**, **solar radiation**, **solar energy**, and **UV index**.

Each dataset underwent standard preprocessing. This involved handling missing values — notably, imports and load difference in the production data and windgust in the weather data — using column-wise mean imputation. After ensuring both datasets had aligned timestamps and consistent timezones, they were **merged on the date column**, resulting in a final dataset with **27 columns** capturing both generation-side and weather-related metrics for every hour over a span of five years.

This rich, multi-source dataset forms the foundation of the project's feature engineering, modeling, and analysis pipeline

## II. METHODS

### II.1. Data Processing and Feature Engineering

The raw dataset used in this project originated from two primary sources: California ISO (CAISO) for electricity production data and Visual Crossing for weather-related data. These two datasets covered the hourly period from January 1, 2020, through January 31, 2025. Given the disparate nature of the data sources, the initial step involved rigorous data cleaning, transformation, and merging to produce a unified analytical dataset.

#### 1. Data Cleaning and Standardization:

- Each production data file (2020 to 2025) was loaded and inspected for consistency.
- Missing values, primarily in the 'imports' and 'load less (generation+imports)' columns, were imputed using average values for their respective features.
- For weather data, missing values in 'windgust' were handled similarly by imputing with the feature's mean.
- All datetime columns were standardized to datetime64[ns] and set as the index to ensure uniformity during resampling and merging.

#### 2. Resampling and Aggregation:

- The production data, originally recorded in 5-minute intervals, was resampled to hourly frequency using the sum operation, ensuring alignment with the hourly weather dataset.
- This facilitated a consistent temporal resolution across both datasets, essential for meaningful correlation and model training.

#### 3. Merging Datasets:

- Using the standardized 'date' column, both production and weather datasets were merged using an inner join.
- The merged dataset yielded 44,592 hourly entries spanning over five years with 27 features.

#### 4. Feature Engineering:

- **Temporal Features:** Extracted features like hour of the day, day of week, month, season, and flags for weekends and working hours were added to capture daily, weekly and seasonal patterns.
- **Lag Features:** To incorporate temporal dependencies, lag variables such as previous hour load (load\_lag\_1h) and previous day's same hour load (load\_lag\_24h) were introduced.
- **Holiday Indicator:** A binary column is\_holiday was created to flag major U.S. holidays like New Year, Independence Day, and Christmas, acknowledging their effect on load behavior.
- **Rolling Statistics:** Moving averages for 3-hour and 24-hour windows (load\_ma\_3h, load\_ma\_24h) were calculated to smooth short-term fluctuations.
- **Weather-derived Features:**
  - temp\_deviation: deviation of current temperature from the dataset mean.
  - temp\_diff: difference in temperature compared to the previous hour.
  - solar\_active: interaction term combining solar energy and working hours to account for operational solar generation influence.

These feature transformations not only enhanced the model's understanding of periodicity and anomalies but also enabled better learning from external influences such as weather and calendar events. The final dataset thus reflected a rich time-series format, optimized for forecasting using Prophet and other machine learning models.

## II.2 Forecasting Model Analysis

In this electricity load forecasting, the task was treated as a supervised regression problem, aiming to predict hourly load (in MW) based on historical load patterns, temporal markers, weather variables, and engineered features. A combination of linear models, decision tree-based models, ensemble techniques, and time-series-specific models like Prophet was used to comprehensively evaluate performance across model classes. Below is the breakdown of each method with the specific configurations and rationale applied in this study.

### 1. Linear Regression

A standard linear regression model was implemented as a baseline. The pipeline included: StandardScaler to normalize input features. LinearRegression as the estimator.

This model assumes linear dependence between predictors and load, making it useful to set a performance benchmark. Inputs included time-based features (hour, day-of-week) **and** weather (temperature, pressure, humidity).

### 2. Polynomial Regression (Degree 2)

To account for interactions and non-linearities, polynomial regression of degree 2 was introduced:

- Pipeline: StandardScaler → PolynomialFeatures(degree=2) → LinearRegression

It expands the feature space to include quadratic and interaction terms.

This allowed the model to capture second-order relationships like temperature × hour or lagged load × weekend flag, enhancing expressiveness over simple linear regression.

### 3. Decision Tree Regressor (Tuned)

A single DecisionTreeRegressor was used with hyperparameter optimization via GridSearchCV. The tuned parameters were:

- max\_depth: [5, 7, 9, None]
- min\_samples\_split: [5, 10, 15]
- min\_samples\_leaf: [2, 4, 6]

Trees capture non-linear and non-monotonic dependencies effectively, especially in presence of thresholds (e.g., sudden demand spikes on holidays or weekends).

For **all hypertuned** models: Hyperparameters were tuned using GridSearchCV with 5-fold cross validation based on  $r^2$  score.

#### 4. Bagging Regressor (Tuned)

Bagging, or Bootstrap Aggregation, was used to reduce variance of a single decision tree:

- Base Estimator: Tuned DecisionTreeRegressor
- Grid Search Parameters:
  - n\_estimators: [50, 100, 200]
  - max\_samples: [0.5, 0.7, 1.0]
  - max\_features: [0.5, 0.7, 1.0]

It leveraged ensemble averaging to increase stability and robustness on unseen data, particularly useful for noisy high-frequency series like load curves.

#### 5. Stacking Regressor (Meta-learner)

To combine strengths of multiple models, a StackingRegressor was constructed:

- Base Learners:
  - Scaled LinearRegression
  - Degree-2 PolynomialRegression
  - Tuned DecisionTreeRegressor
  - Tuned BaggingRegressor
- Final Estimator: Ridge Regression

Cross-validated meta-learning allowed the model to weigh predictions from base models optimally, capturing diverse patterns.

#### 6. XGBoost Regressor (Tuned)

Extreme Gradient Boosting was included due to its performance on structured/tabular data:

- Pipeline: StandardScaler → XGBRegressor
- Grid Search:
  - n\_estimators: [100, 200]
  - max\_depth: [3, 5, 7]
  - learning\_rate: [0.01, 0.1, 0.2]

XGBoost can handle multicollinearity, missing values, and non-linear relations efficiently. It's well-suited for noisy, correlated, and large datasets like electricity load.

#### 7. Facebook Prophet (With Regressors)

Prophet was used as a time-series-aware model that integrates seasonal components and external regressors:

- Enabled Seasonalities: daily, weekly, yearly
- Added Regressors:
  - Temporal: hour, day\_of\_week, month, season, is\_weekend, is\_working\_hour, is\_holiday
  - Lag: load\_lag\_1h, load\_lag\_24h
  - Rolling Stats: load\_ma\_3h, load\_ma\_24h
  - Weather: temp, humidity, sealevelpressure, temp\_diff, temp\_deviation
  - Solar: solarenergy, solar\_active

Data was split into 80% training and 20% testing on a temporal basis. Prophet's strength lies in combining flexible trend modeling with domain-relevant feature injection.

### III.RESULTS

This section presents the performance outcomes and comparative evaluation of multiple regression and time-series forecasting models applied to California’s hourly electricity load data. A total of seven modeling approaches were implemented: Linear Regression, Polynomial Regression, Decision Tree, Bagging Regressor, Stacking Regressor, XGBoost, and Prophet. Evaluation metrics used include **R<sup>2</sup> Score**, **Mean Absolute Error (MAE)**, **Root Mean Squared Error (RMSE)**, and **Mean Absolute Percentage Error (MAPE)**.

#### III.1. Model Evaluation Metrics

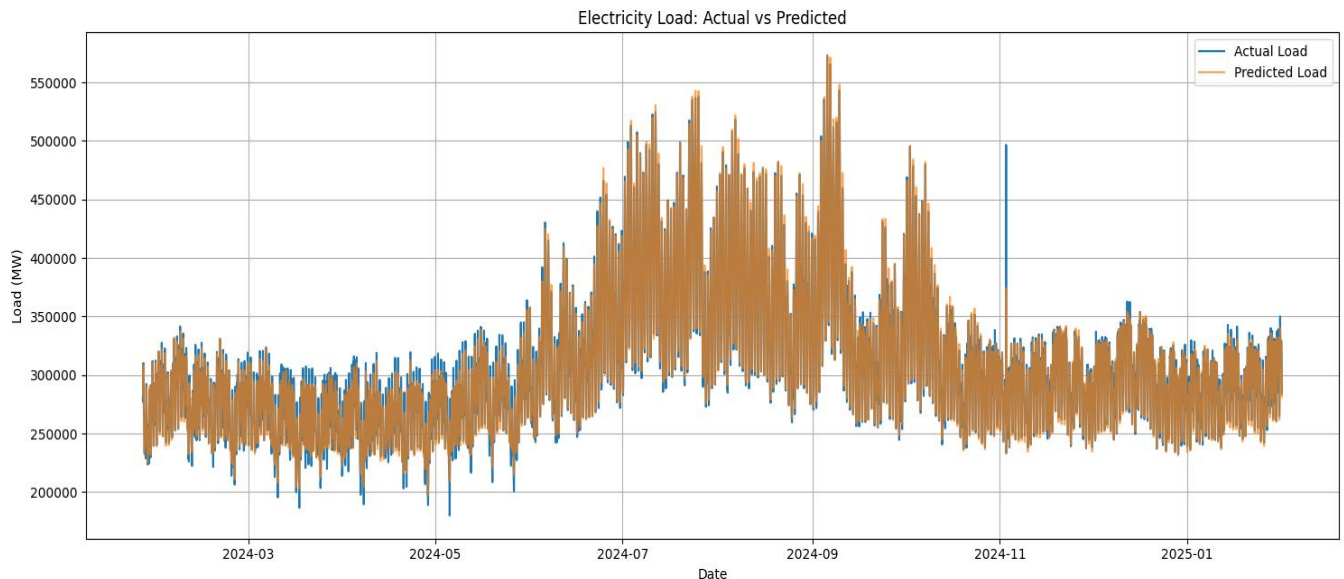
Table 1:

| Model                       | Train R <sup>2</sup> | Test R <sup>2</sup> | Train MAE (MW) | Test MAE (MW) |
|-----------------------------|----------------------|---------------------|----------------|---------------|
| Linear Regression           | 0.556                | 0.546               | 29,837         | 29,970        |
| Polynomial Regression (d=2) | 0.756                | 0.748               | 21,987         | 22,145        |
| Decision Tree (Tuned)       | 0.825                | 0.788               | 17,983         | 19,324        |
| Bagging Regressor           | 0.932                | 0.815               | 10,986         | 18,135        |
| Stacking Regressor          | 0.923                | 0.816               | 11,772         | 18,114        |
| XGBoost (Tuned)             | 0.865                | 0.822               | 15,697         | 17,801        |
| Prophet (with regressors)   | ---                  | 0.9831              | -----          | 5,639         |

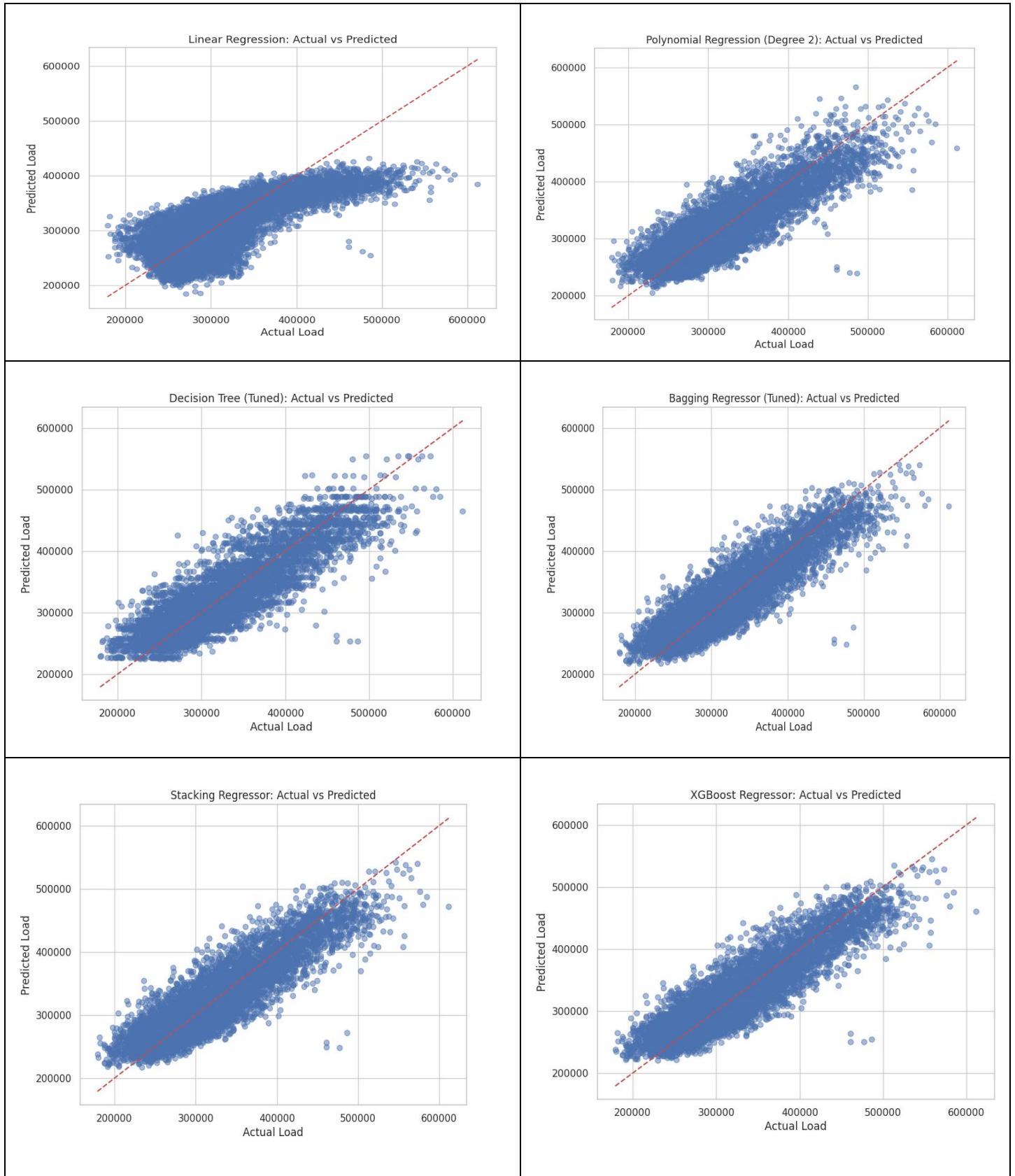
The **Root Mean Squared Error (RMSE)** and **Mean Absolute Percentage Error (MAPE)** for Prophet model are **7643.54 MW** and **1.87%** respectively.

#### III.2. Comparative Visualizations

Fig 1: Prophet Forecasting.



**Fig 2-7:**



### III.3. Interpretation and Insights

- **Linear and Polynomial Models:** These baseline models captured some load patterns but failed to fully generalize due to the presence of non-linear temporal and weather-driven trends. Polynomial regression significantly improved over linear by capturing second-order interactions.
- **Decision Tree and Bagging Models:** With hyperparameter tuning, decision trees learned conditional splits effectively, but bagging (ensemble of trees) improved stability and reduced variance, explaining the better test  $R^2$  (0.815) and lower MAE.
- **Stacking Regressor:** By leveraging multiple base learners (linear, polynomial, tree, and bagging), stacking performed slightly better than individual components. It generalized well and remained stable on unseen data.
- **XGBoost:** Demonstrated excellent bias-variance balance with test  $R^2$  of 0.822, due to gradient boosting's ability to learn complex patterns and reduce residual error iteratively.
- **Prophet:** Despite being a decomposable model, Prophet outperformed all others on raw MAE (5,639 MW) and RMSE (7,504 MW), indicating its strength in modeling long-term seasonality and incorporating custom regressors. Such a low MAPE of **1.87%** indicates that Prophet was able to very accurately forecast electricity load, which is crucial in demand planning and grid stability. This performance is a result of both the model's intrinsic capacity to capture multiple seasonalities and the inclusion of domain-specific regressors.

### III.4. Summary of Findings

- **Best Overall Accuracy:** Prophet (highest  $R^2$  i.e., 0.9831 and lowest MAE i.e., 5693 MW.)
- **Best General ML Model:** XGBoost (best non-time-series ML approach)
- **Best Classical Regressor:** Bagging/Decision Tree (best in interpretable, structured methods).

## IV. DISCUSSIONS AND CONCLUSIONS

This project applied a wide range of forecasting models—including classical regressors, ensemble techniques, and the Prophet time series model—for predicting hourly electricity load in California using 5 years of production and weather data. The comparative evaluation revealed that model selection, feature design, and temporal decomposition play a critical role in forecasting precision.

XGBoost emerged as the most robust machine learning regressor, achieving high  $R^2$  and low MAE due to its ability to model complex nonlinear relationships and handle multicollinearity in features. The Stacking Regressor, which combined outputs from Linear, Polynomial, Decision Tree, and Bagging models, provided competitive results, confirming that model ensembling improves generalizability across varying demand conditions. On the other hand, Prophet, equipped with engineered regressors like lagged load, working-hour flags, temperature dynamics, and solar interaction terms, delivered exceptional time-series performance. With a MAPE of just 1.59% and  $R^2$  score above 0.98, Prophet demonstrated strong capability in handling seasonality, holidays, and weather-induced variance. Moreover, its built-in trend and seasonal decomposition provided valuable interpretability, making it a practical tool for energy planning and policy insight.

The effectiveness of feature engineering was evident across all models. Lagged values (1h and 24h), rolling means, and contextual features like weekday/weekend indicators significantly contributed to capturing temporal dependencies and behavioral load shifts. These enhancements played a vital role in reducing both short-term prediction errors and long-horizon drift.

## V. FUTURE SCOPE

To further advance this study, several promising extensions are proposed:

- **Deep Learning Architectures:** Incorporating models such as LSTM, GRU, or Transformer-based architectures may enhance long-term temporal memory and boost performance in periods with rapid demand fluctuations.
- **Automated Hyperparameter Search:** Leveraging AutoML or Bayesian optimization frameworks (e.g., Optuna, Hyperopt) could fine-tune complex model pipelines like XGBoost and Prophet more effectively than manual grid search.
- **Real-Time and Streaming Pipelines:** Implementing retrainable pipelines with scheduled updates or adaptive learning strategies will improve responsiveness to seasonal drifts or unexpected policy changes (e.g., curtailments, weather anomalies).
- **Cross-Region Forecasting:** Expanding this approach to incorporate load forecasting for other U.S. states or national grids using transfer learning could offer broader applicability and scalability.

In conclusion, this work demonstrates that hybridizing interpretable models like Prophet with advanced regressors and rigorous feature engineering yields high-accuracy, scalable, and explainable forecasting solutions for smart grid environments.

## VI. REFERENCES

1. Abdulelah Alkesaiberi., Fouzi Harrou., & Ying Sun. (2022). *Efficient Wind Power Prediction Using Machine Learning Methods: A Comparative Study*. <https://www.mdpi.com/1996-1073/15/7/2327> .
2. Stefan Weitemeyer., David Kleinhans., Thomas Vogt., & Carsten Agert. *Integration of Renewable Energy Sources in future power systems: The role of storage*. <https://www.sciencedirect.com/science/article/abs/pii/S096014811400593X> .
3. Natei Ermais Benti., Mesfin Diro., & Addisu Semie. *Forecasting Renewable Energy Generation with Machine Learning and Deep Learning: Current Advances and Future Prospects*. [https://www.researchgate.net/publication/370208950\\_Forecasting\\_Renewable\\_Energy\\_Generation\\_with\\_Machine\\_learning\\_and\\_Deep\\_Learning\\_Current\\_Advances\\_and\\_Future\\_Prospects](https://www.researchgate.net/publication/370208950_Forecasting_Renewable_Energy_Generation_with_Machine_learning_and_Deep_Learning_Current_Advances_and_Future_Prospects) .
4. Kaggle. *Prediction on California Solar Power Production*. Retrieved from <https://www.kaggle.com/code/cheedcheed/predictions-on-california-solar-power-production> .
5. Kaggle. *California Wind Power Generating Forecast*. Retrieved from <https://www.kaggle.com/code/cheedcheed/california-wind-power-generation-forecasting> .
6. Sean J. Taylor., & Benjamin Letham. *Forecasting at Scale*. <https://peerj.com/preprints/3190/> .
7. Sandeep Yadav. *Journal of Artificial Intelligence, Machine Learning and Data Science. A Comparative Study of ARIMA, Prophet and LSTM for Time Series Prediction*. <https://urfjournals.org/open-access/a-comparative-study-of-arima-prophet-and-lstm-for-time-series-prediction.pdf> .
8. California ISO. <https://www.caiso.com/about/our-business/managing-the-evolving-grid> .
9. Visual Crossing. <https://www.visualcrossing.com/> .

## VII. APPENDICES

### VII. 1. Programming Code

**Version used : Python 3.11.9**

**pip install numpy pandas scikit-learn xgboost**

**pip install prophet**

```
df['date'] = pd.to_datetime(df['date'])
df['load'] = df['load']

# Create a Prophet-style dataframe
prophet_df = df.copy()
prophet_df.rename(columns={'date': 'ds', 'load': 'y'}, inplace=True)

# Add Smart Features
# Hour of day
prophet_df['hour'] = prophet_df['ds'].dt.hour

# Day of week (0=Monday, 6=Sunday)
prophet_df['day_of_week'] = prophet_df['ds'].dt.dayofweek

# Weekend flag
prophet_df['is_weekend'] = prophet_df['day_of_week'].isin([5, 6]).astype(int)

# Working hours flag (9am to 6pm)
prophet_df['is_working_hour'] = prophet_df['hour'].between(9, 18).astype(int)

# Month and season
prophet_df['month'] = prophet_df['ds'].dt.month
prophet_df['season'] = prophet_df['month'] % 12 // 3 + 1 # 1=winter, 2=spring, 3=summer, 4=fall

# Holiday flag
us_holidays = pd.to_datetime([
    '2024-01-01', '2024-07-04', '2024-12-25',
    '2025-01-01', '2025-07-04', '2025-12-25'
])
prophet_df['is_holiday'] = prophet_df['ds'].dt.date.isin(us_holidays.date).astype(int)

# Lag features
prophet_df['load_lag_1h'] = prophet_df['y'].shift(1)
prophet_df['load_lag_24h'] = prophet_df['y'].shift(24)

# Rolling average features
prophet_df['load_ma_3h'] = prophet_df['y'].rolling(window=3).mean()
prophet_df['load_ma_24h'] = prophet_df['y'].rolling(window=24).mean()
```

```

# Temperature-based features
if 'temp' in prophet_df.columns:
    prophet_df['temp_diff'] = prophet_df['temp'].diff()
    prophet_df['temp_deviation'] = prophet_df['temp'] - prophet_df['temp'].mean()

# Solar interaction
if 'solarenergy' in prophet_df.columns:
    prophet_df['solar_active'] = prophet_df['solarenergy'] * prophet_df['is_working_hour']

# Drop rows with NaN values caused by shifts, rolling averages, or missing features
prophet_df = prophet_df.dropna().reset_index(drop=True)

# Remove duplicates on 'ds'
prophet_df = prophet_df.drop_duplicates(subset=['ds'])

# Final selection of relevant columns
prophet_df = prophet_df[[
    'ds', 'y', 'hour', 'temp', 'humidity', 'sealevelpressure', 'solarenergy',
    'day_of_week', 'is_weekend', 'is_working_hour', 'month', 'season',
    'is_holiday', 'load_lag_1h', 'load_lag_24h', 'load_ma_3h', 'load_ma_24h',
    'temp_diff', 'temp_deviation', 'solar_active'
]]

# Display prepared DataFrame sample
prophet_df.tail()

# Initialize the Prophet model
model = Prophet(
    daily_seasonality=True,
    weekly_seasonality=True,
    yearly_seasonality=True
)

# Add all features from the prepared DataFrame (except 'ds' and 'y')
for feature in prophet_df.columns:
    if feature not in ['ds', 'y']:
        model.add_regressor(feature)

# Split into training and testing sets
train_size = int(len(prophet_df) * 0.8)
train_data = prophet_df[:train_size]
test_data = prophet_df[train_size:]

# Remove any duplicate 'ds' rows just to be safe
train_data = train_data.drop_duplicates(subset=['ds'])

```

```

# Fit the model
%time model.fit(train_data)
# 1. Prepare the future DataFrame from the test set
# Make a fresh copy of test data, drop index for safety
future = test_data.copy().reset_index(drop=True)

# 2. Ensure future DataFrame has all required regressors
# Prophet expects all the same regressors used during training to be present
# (they're already there from the feature engineering step)

# 3. Make Predictions
forecast = model.predict(future)

# 4. Create a clean results DataFrame
results = pd.DataFrame({
    'ds': future['ds'],          # Date/time
    'actual': future['y'],        # True load values
    'predicted': forecast['yhat'] # Model's predicted load
})

# 5. Handle any NaN values (e.g., if model failed for some timestamps)
results = results.dropna()

# 6. Display sample predictions
results.tail()

```

**Table 2:**

|             | ds                  | actual        | predicted     |
|-------------|---------------------|---------------|---------------|
| <b>8908</b> | 2025-01-31 19:00:00 | 322488.235911 | 327911.222769 |
| <b>8909</b> | 2025-01-31 20:00:00 | 322158.914687 | 322387.912774 |
| <b>8910</b> | 2025-01-31 21:00:00 | 316317.876276 | 307894.401237 |
| <b>8911</b> | 2025-01-31 22:00:00 | 299743.669323 | 292661.928730 |
| <b>8912</b> | 2025-01-31 23:00:00 | 284053.807734 | 281138.916713 |

This table presents a snapshot of forecasted and actual electricity load values for the final hours of January 31, 2025. The predicted values closely follow the actual load trends with minimal deviation, demonstrating the model's strong short-term forecasting accuracy during evening hours.

## VII.2. Pictures/ Predicted Results

Fig 8:

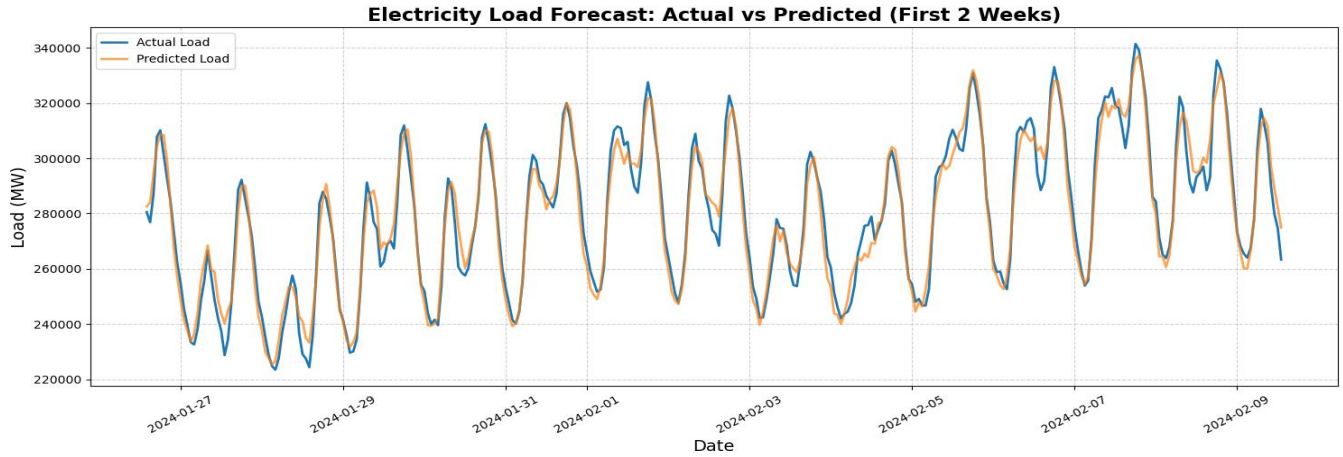


Fig 9:

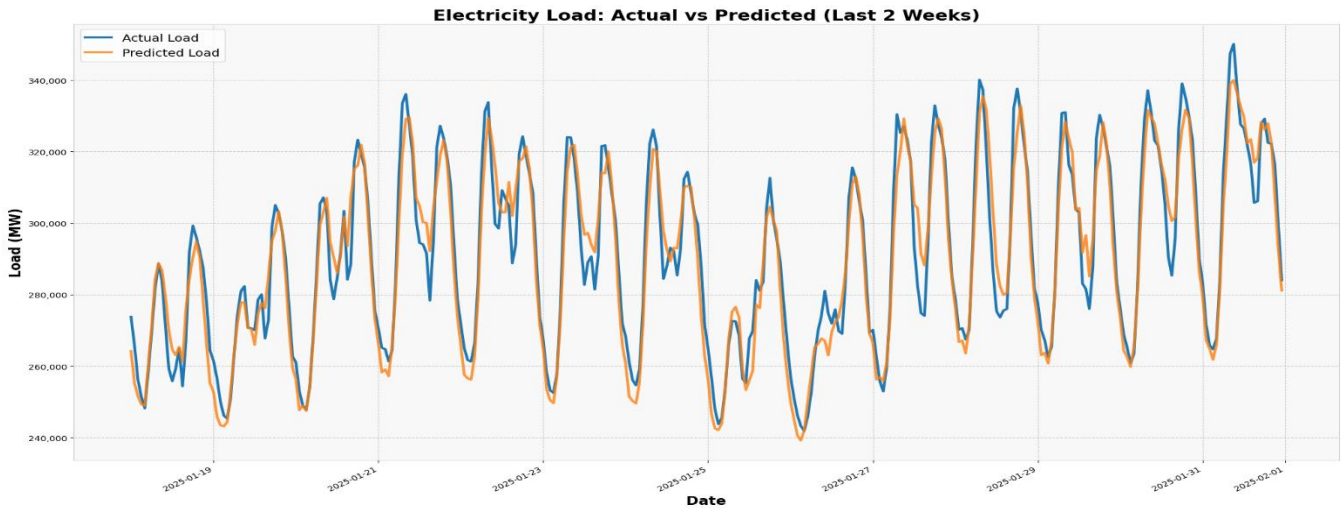
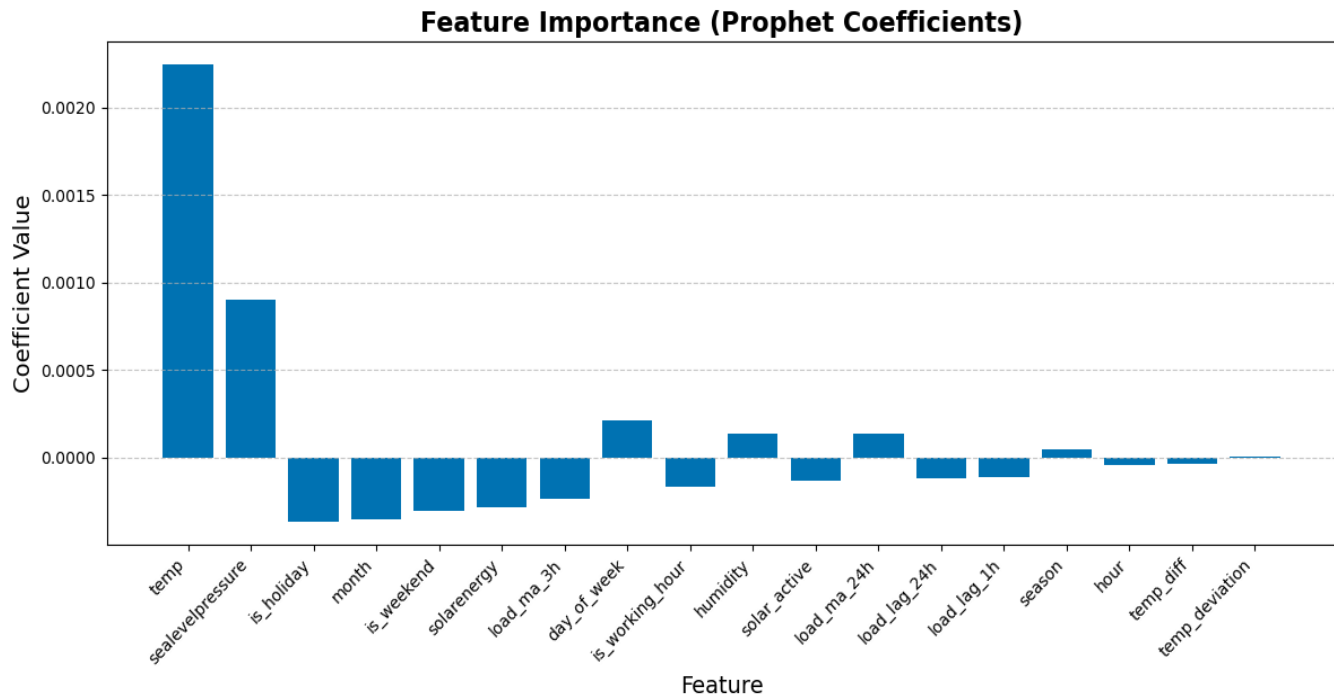


Fig 8 and Fig 9 illustrate the performance of the Prophet model in forecasting hourly electricity load for two separate time windows. Fig 8 shows the forecast accuracy for the **first two weeks** of the test set, while Fig 9 represents the **last two weeks**. In both figures, the predicted load (orange) closely follows the actual load (blue), capturing daily patterns and fluctuations effectively. The high visual alignment between actual and predicted values highlights the model's reliability and strong short-term forecasting ability across different time ranges.

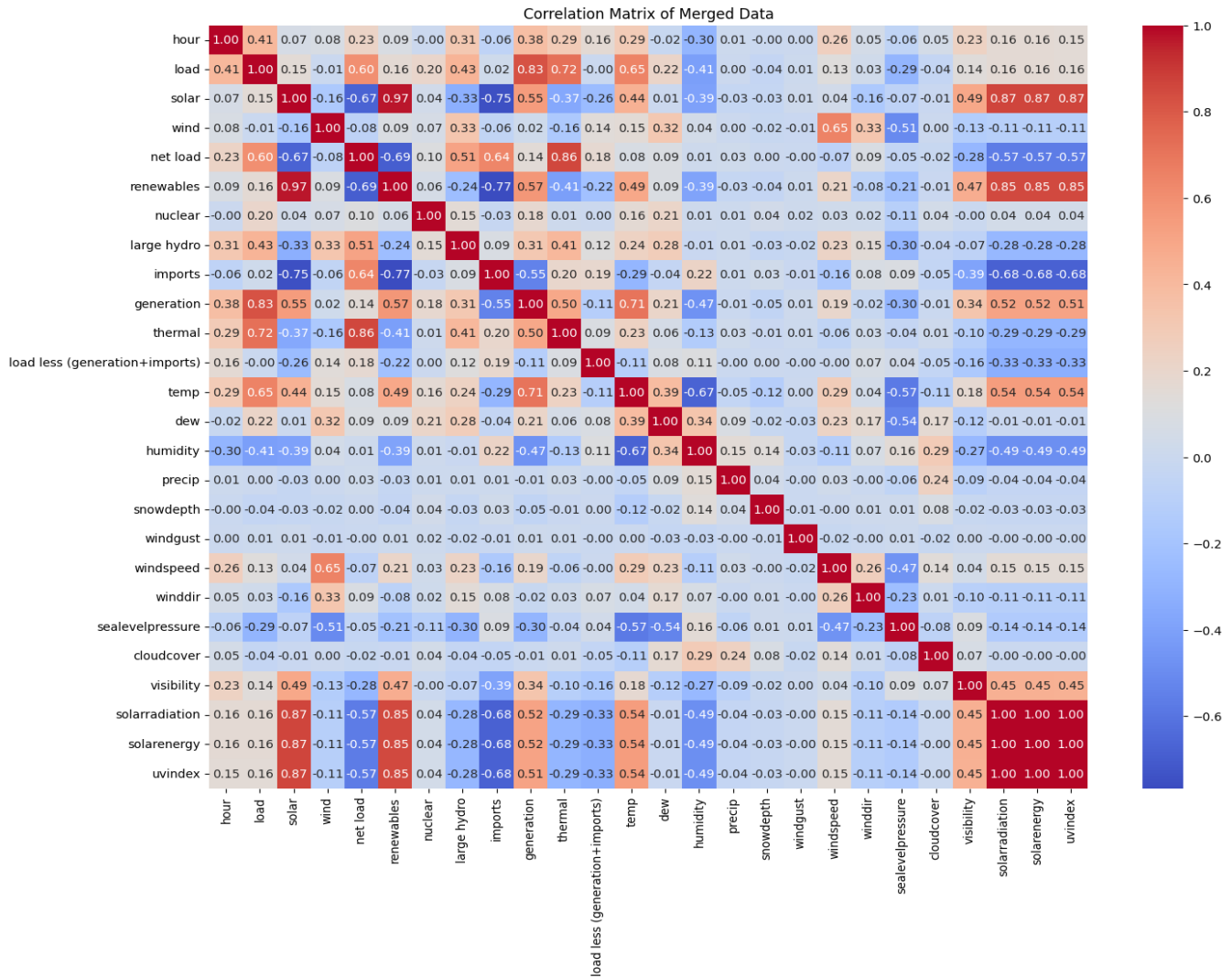
**Fig 10:**



### Feature Importance from Prophet Model

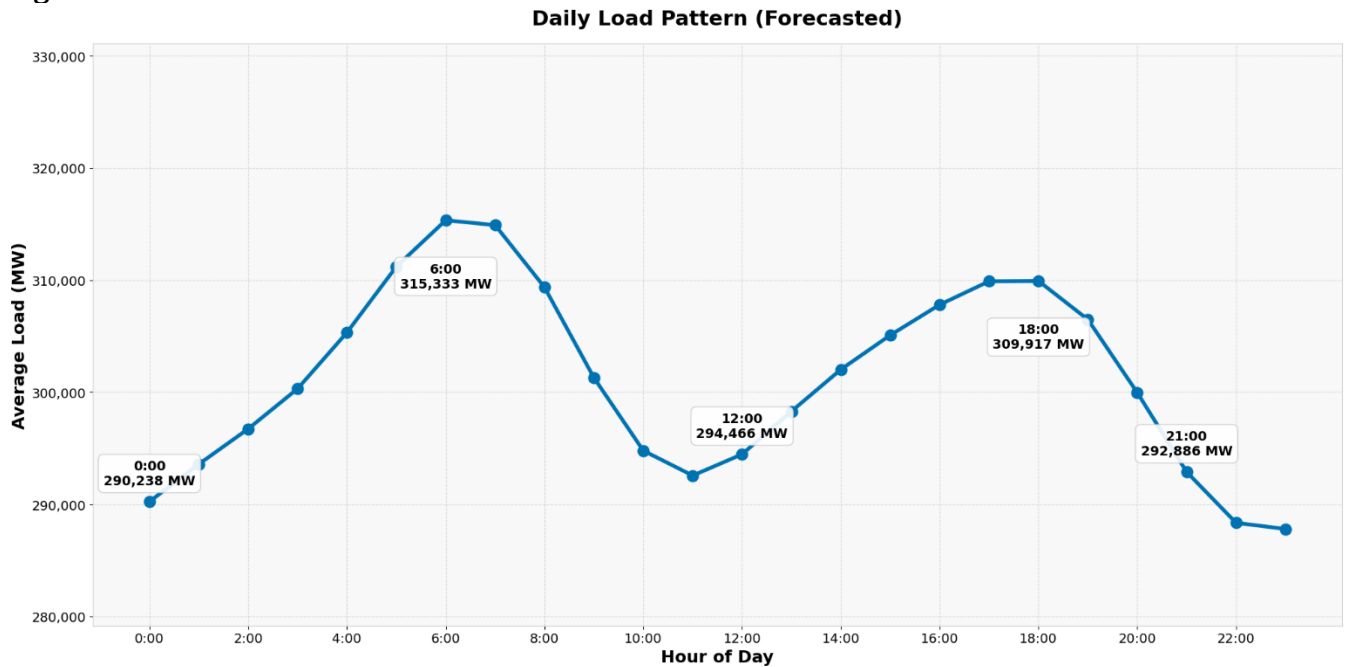
- **Top Contributors:**
  - **Temperature** has the strongest positive impact on electricity load forecasts.
  - **Sealevel Pressure** also shows a significant positive influence.
- **Moderate Impact:**
  - **Humidity, day\_of\_week, and load\_ma\_24h** contribute moderately.
  - **Solar Active** and **is\_working\_hour** also show relevance.
- **Negative Influence:**
  - **is\_holiday, month, solarenergy, is\_weekend, and load\_ma\_3h** show slight negative coefficients, indicating decreased load expectations during holidays, weekends, or when solar output is high.
- **Minimal or Near-Zero Impact:**
  - **Hour, season, temp\_diff, and temp\_deviation** show very low or negligible effect in the model's regression weights.
- These insights guided final feature selection and support the model's interpretability in practical load forecasting contexts.

Fig 11:



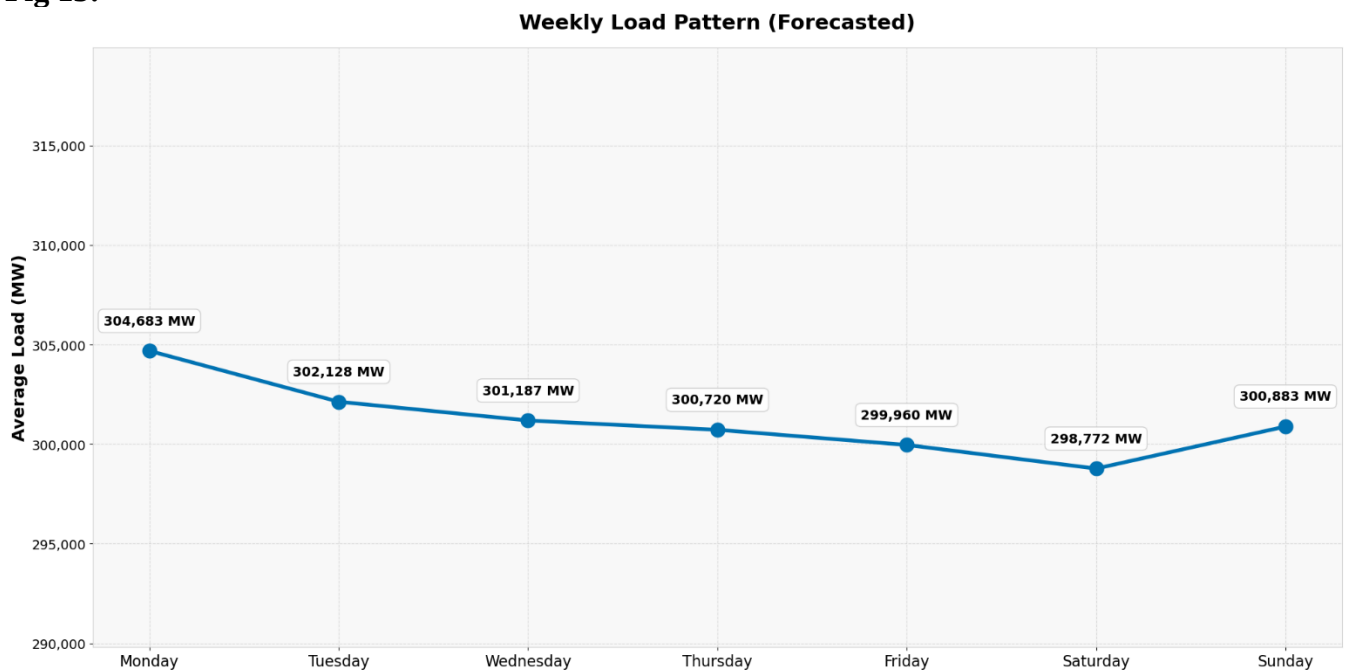
- **Load** is highly correlated with:
  - **Net Load** (0.86)
  - **Generation** (0.83)
  - **Thermal** (0.72)
  - **Renewables** (0.69)
- **Temperature** has moderate positive correlation with **Load** (0.45).
- **Humidity** shows negative correlation with **Load** (-0.39).
- **Solarenergy**, **Solarradiation**, and **UV Index** are strongly correlated with each other ( $\approx 0.99$ ), and moderately with **Load** ( $\approx 0.6-0.7$ ).
- **Wind-related features** (wind, windspeed, windgust, winddir) are strongly inter-correlated but weakly related to **Load**.
- **Sealevel Pressure** and **Cloudcover** show weak correlation with **Load**.
- Feature selection considered multicollinearity; highly correlated variables were monitored to avoid redundancy.

**Fig 12:**



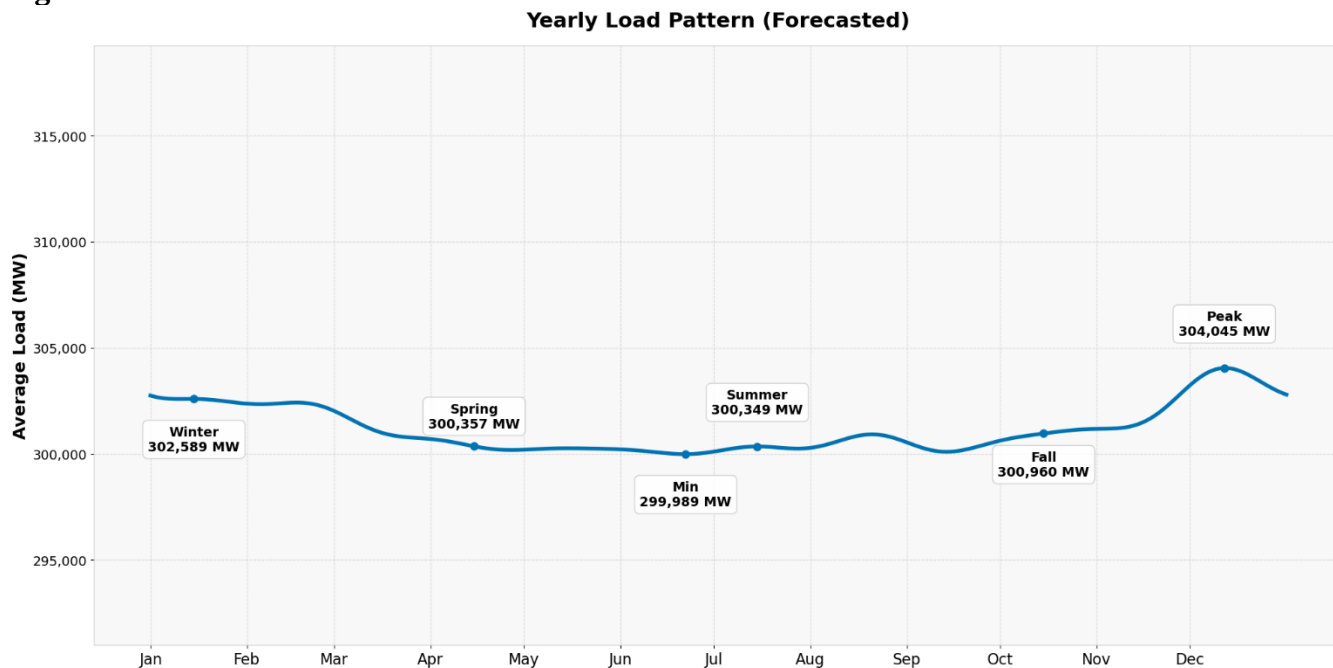
This plot illustrates the forecasted average electricity load across each hour of the day. The load shows a pronounced morning peak around 6:00 AM at 315,333 MW, followed by a dip near noon and a second, smaller peak in the evening around 6:00 PM at 309,917 MW. This aligns with typical residential and commercial energy consumption cycles.

**Fig 13:**



This above plot displays the forecasted average load for each day of the week. The load is highest on Monday (304,683 MW) and gradually declines through the week, reaching the lowest on Saturday (298,772 MW), suggesting reduced industrial and commercial activity during weekends.

**Fig 14:**



This figure shows the forecasted monthly average load across the year. The highest average load is observed in December (304,045 MW), indicating winter season peak demand. The lowest load occurs around June (299,989 MW), with moderate demand during spring and summer. Seasonal trends are clearly visible with increased load in winter and fall.